PostgreSQL vs MongoDB: Architecture Comparison for SaaS Platforms

Synthetic Internal Reference Document — Version 1.0 — October 2024

1. Introduction

This document provides an architectural comparison of PostgreSQL (relational) and MongoDB (document-oriented) for multi-tenant SaaS platform deployments. It evaluates data model design, multi-tenant isolation strategies, replication and high availability, indexing and query performance, and operational considerations. The goal is to support evidence-based technology selection decisions aligned with the organization's architecture guidelines.

2. Data Model and Schema Design

2.1 PostgreSQL — Relational Model

PostgreSQL uses a strictly typed relational schema with tables, columns, foreign keys, and constraints. For SaaS applications, this provides strong data integrity through CHECK constraints, NOT NULL enforcement, and referential integrity via foreign keys. Schema migrations are explicit and version-controlled. PostgreSQL also supports JSONB columns for semi-structured data, offering flexibility when needed without abandoning the relational model. Complex multi-table JOINs are a core strength, with the query planner optimizing execution paths across indexes and statistics.

2.2 MongoDB — Document Model

MongoDB stores data as flexible BSON documents within collections. There is no enforced schema by default, though schema validation rules can be applied at the collection level. The document model naturally maps to object-oriented application code, reducing impedance mismatch. Embedding related data within a single document eliminates many JOIN operations, improving read performance for document-centric access patterns. However, deeply nested documents can lead to update complexity and working set inflation.

3. Multi-Tenant Isolation Strategies

3.1 PostgreSQL Approaches

PostgreSQL supports multiple multi-tenant isolation models:

Schema-per-tenant: Each tenant gets a dedicated schema within a shared database. This provides logical isolation while sharing compute and connection resources. Row-Level Security (RLS) policies enforce access boundaries at the database engine level, preventing cross-tenant data leakage even if application-level bugs exist. This approach scales well to 1,000–5,000 tenants before schema management overhead becomes significant.

Shared-table with tenant_id: All tenants share the same tables, distinguished by a tenant_id column. RLS policies filter rows automatically. This is the most resource-efficient model but requires careful index design (composite indexes including tenant_id) and thorough testing of the RLS policy coverage.

Database-per-tenant: Maximum isolation but highest resource cost. Each tenant database has its own connection pool, backup schedule, and can be independently scaled or migrated.

3.2 MongoDB Approaches

MongoDB's primary multi-tenant isolation models include:

Database-per-tenant: Each tenant has a separate database within the same cluster. This provides strong namespace isolation and simplifies per-tenant backup/restore. However, MongoDB Atlas imposes limits on the number of databases per cluster, and connection pooling overhead grows linearly with tenant count. Beyond 500 tenants, connection pool pressure becomes a significant operational concern.

Collection-per-tenant: Tenants share a database but have separate collections. This reduces connection overhead but complicates cross-tenant aggregation queries.

Shared collection with tenant field: Similar to PostgreSQL's shared-table approach but without native row-level security. Isolation relies entirely on application-layer query filtering, which increases the risk of data leakage through developer error.

4. Replication and High Availability

Feature	PostgreSQL	MongoDB
Replication Type	Streaming (physical) + Logical	Replica Sets (oplog-based)
Automatic Failover	Via Patroni / pg_auto_failover	Built-in (primary election)
Read Scaling	Read replicas (async/sync)	Secondary reads (eventual consistency)
Multi-Region	Logical replication to remote regions	Zone-sharded, geo-distributed clusters
Failover Time	10–30 seconds (with Patroni)	5–15 seconds (replica set election)
Split-Brain Protection	Requires external consensus (etcd)	Built-in majority write concern

MongoDB's built-in replica set architecture provides simpler high-availability setup compared to PostgreSQL, which traditionally requires third-party tooling (Patroni, Stolon, or pg_auto_failover) for automated primary election. However, PostgreSQL's logical replication offers more flexible cross-version and cross-platform data distribution capabilities.

5. Indexing and Query Performance

PostgreSQL's query planner is widely regarded as one of the most sophisticated in the database industry. It supports B-tree, Hash, GiST, SP-GiST, GIN, and BRIN index types, enabling optimization for diverse access patterns. Partial indexes, expression indexes, and covering indexes (INCLUDE clause) further reduce I/O for targeted queries. For multi-tenant deployments, composite indexes with tenant_id as the leading column are essential for performance.

MongoDB supports B-tree indexes, compound indexes, multi-key indexes (for array fields), text indexes, and geospatial indexes. The WiredTiger storage engine provides document-level concurrency control. MongoDB's aggregation pipeline offers expressive data transformation capabilities, though complex multi-collection aggregations via $lookup can be significantly slower than PostgreSQL's native JOIN optimization.

For read-heavy workloads with simple document lookups, MongoDB typically achieves 10-20% lower latency due to its storage engine's alignment with document access patterns. For write-heavy workloads with complex transactions spanning multiple entities, PostgreSQL's MVCC implementation and mature lock management provide more predictable performance under contention.

6. ACID Transaction Support

PostgreSQL has provided full ACID transaction support since its inception. Transactions can span any number of tables, schemas, and operations with complete isolation level support (Read Uncommitted, Read Committed, Repeatable Read, Serializable). This is a foundational requirement for financial and billing operations in SaaS platforms.

MongoDB introduced multi-document ACID transactions in version 4.0 (2018) and has progressively improved their performance and scope through version 7.0. Transactions now support operations across multiple documents, collections, and databases within a replica set. However, transaction performance degrades when operations span multiple shards in a sharded cluster, with latency increasing by approximately 20-40% compared to single-shard transactions. For most SaaS use cases that can co-locate tenant data on a single shard, this limitation is manageable.

7. Summary and Considerations

The choice between PostgreSQL and MongoDB for a multi-tenant SaaS platform depends on the specific workload characteristics, team expertise, and compliance requirements:

Choose PostgreSQL when: The workload involves complex relational queries, strong consistency is critical, ACID compliance is mandatory across all operations, the team has relational database expertise, and cost optimization at scale is a priority.

Choose MongoDB when: The data model is predominantly document-oriented, read-heavy workloads dominate, schema flexibility is important for rapid iteration, built-in horizontal scaling is required from day one, and the team has NoSQL experience.

Both databases are mature, well-supported platforms with robust managed service options. The decision should be informed by representative performance benchmarks, total cost of ownership

analysis, and alignment with the organization's architectural guidelines and compliance requirements.